Dossier technique — Plateforme RH NovaCorp

Oct 28, 2026 · @nael

1. Contexte et périmètre

La plateforme remplace le traitement manuel des demandes RH de NovaCorp (120 personnes) par un cycle unique : soumission, contrôle, approbation, notification, avec historique et délais mesurables.

Aujourd'hui, les demandes arrivent par email, sont recopiées dans un tableur, validées par retour d'email sans traçabilité, et relancées au téléphone. Personne ne mesure le délai réel d'une demande.

Le périmètre couvre les six fonctions du cahier des charges :

1. Un salarié soumet une demande (congés, télétravail, matériel, formation) via un formulaire.
2. Le workflow route la demande vers le bon valideur selon le type et le montant ou la durée (manager, puis RH, puis DRH au-delà d'un seuil).
3. L'agent IA contrôle la demande (dates cohérentes, solde de congés suffisant, doublons) et prépare une synthèse pour le valideur.
4. Le valideur approuve ou refuse en un clic, avec traçabilité et délai.
5. Le salarié est notifié à chaque changement de statut.
6. Le service RH dispose d'un tableau de bord en temps réel.

Toutes les données du projet sont fictives (RGPD).

2. Grille de décision low-code (Q1–Q5)

> À compléter par l'équipe avec la grille vue en séance 1 : les cases ci-dessous donnent la justification propre à ce projet, pas la réponse officielle.

Question	Réponse pour ce projet	Choix
Q1	*à reprendre de la grille de séance 1*	Supabase (données, auth, RLS)
Q2	*idem*	n8n (notifications, orchestration IA)

Question	Réponse pour ce projet	Choix
Q3	*idem*	Code SQL (règles critiques : routage, solde)
Q4	*idem*	React (interface sur mesure)
Q5	*idem*	Le LLM ne décide jamais, il rédige

Principe retenu : le low-code pour ce qui change souvent et se branche sur des services (n8n), le code pour ce qui doit être exact et testé (règles de validation et de solde en SQL, couvertes par `tests/smoke.sql`).

3. Architecture

De	Vers	Ce qui circule
Utilisateur (salarié, manager, RH, DRH)	React + Vite	saisie et validation des demandes
React	Supabase (Postgres, Auth, Realtime)	clé publishable, droits par RLS
Supabase	n8n	Database Webhook à chaque changement sur `requests`
n8n	Supabase	clé `service_role` : `check_request`, `ai_record_review`
n8n	LLM (Gemini)	JSON des contrôles, retour d'une synthèse
n8n	Email (SMTP)	notifications manager et demandeur
Supabase	React	Realtime : l'interface se met à jour seule

- **Supabase** porte les données, l'authentification, les droits (RLS) et les règles critiques (fonctions SQL).
- **React** est le seul point d'entrée utilisateur ; il passe par les fonctions `approve_request` / `reject_request` pour décider.
- **n8n** orchestre les effets de bord (IA, emails) sans jamais décider.

4. Schéma de données

Table	Rôle	Points clés
`profiles`	utilisateurs (liés à `auth.users`)	`role` (employee, manager, hr, drh), `manager_id`
`request_types`	catalogue des demandes	seuils `hr_threshold / drh_threshold`, `threshold_unit` (days ou eur), `needs_dates`, `needs_amount`, `consumes_leave`
`requests`	les demandes	`status`, `required_level` (calculé par trigger), `days_requested` (colonne générée), `decided_at`, `decided_by`
`request_history`	journal des changements d'état	alimenté par trigger, note via `app.note`
`leave_balances`	solde de congés	débité à l'approbation finale
`ai_reviews`	contrôles IA	`verdict`, `checks` (jsonb), `summary`, `tokens_in/out`, `cost_eur`

Statuts : pending → manager_review → (hr_review → drh_review) → approved, ou rejected à n'importe quelle étape.

Routage (`route_level`) : sous le premier seuil le manager suffit, au-delà la RH, au-delà du second seuil la DRH. Exemples seed : congés 10 / 20 jours, matériel 500 / 2000 €, formation 1000 / 5000 €.

Vues : `v_dora_lead_time`, `v_stage_durations`, `v_ai_costs_monthly` (toutes en `security_invoker`, donc soumises à la RLS).

Le schéma est entièrement versionné dans `supabase/migrations/` (6 fichiers), rejouable avec `supabase db push`.

5. Sécurité et RGPD

- **RLS partout** : un salarié ne voit que ses demandes, un manager celles de son équipe (et le solde de ses membres), RH/DRH tout.
- **Aucune écriture directe de décision** : le statut ne se modifie que via les fonctions `approve_request` et `reject_request` (`SECURITY DEFINER`). Elles vérifient le rôle, l'équipe, l'étape en cours, l'interdiction de s'auto-approuver et exigent un motif pour un refus.

- **Création contrôlée** : une demande ne peut être créée que par son auteur et avec le statut `pending` (impossible d'insérer une demande déjà approuvée).
- **Clé `service_role` réservée à n8n** : seules `check_request`, `ai_record_review` et `purge_old_requests` lui sont ouvertes. Le front n'utilise que la clé publishable.
- **Webhook n8n** protégé par un secret en en-tête (Header Auth), à régénérer avant la soutenance.
- **Minimisation** : seules les données utiles à la décision (dates, montant, motif) sont envoyées au LLM, jamais l'email ni d'identifiant personnel.
- **Conservation** : `purge_old_requests(24)` supprime les demandes de plus de 24 mois (à planifier avec n8n).
- **Données fictives uniquement** (comptes de test).

6. Workflows n8n

n8n tourne en Docker, exposé par un tunnel cloudflared (l'URL change à chaque redémarrage). Deux flux :

1. **Notification** : Webhook (Header Auth, appelé par un Database Webhook Supabase sur `requests`) → nœud If (INSERT ou UPDATE) → lecture du type et du manager (nœud Supabase) → Send Email (SMTP Mailtrap) vers le manager à la création, vers le demandeur à la décision.
2. **Contrôle IA** (voir section 7) : Webhook → `check_request` → LLM → `ai_record_review`.

Étape	Déclencheur	Action
1	Création d'une demande (React → Supabase)	Database Webhook vers n8n
2	nœud If : INSERT ou UPDATE ?	INSERT → contrôle IA, UPDATE → notification de décision
3	INSERT	`check_request` → LLM (synthèse) → `ai_record_review` (la demande passe en `manager_review`)
4	Fin du contrôle IA	email au manager
5	UPDATE (décision)	email au demandeur

Le routage manager → RH → DRH n'est **pas** dans n8n : il est dans la base (`route_level`, `approve_request`), donc il reste exact même si n8n est à l'arrêt.

7. Agent IA : contrôle et chiffrage

Principe : les faits sont calculés par la base (déterministe, testé), le LLM ne fait que rédiger. L'IA ne décide jamais : l'humain valide.

Étape	Qui	Ce qui est fait
1. Validation	`check_request(id)` (SQL)	dates cohérentes et non passées, montant présent si requis, solde de congés suffisant, doublons qui se chevauchent, niveau de validation requis
2. Synthèse	LLM (Gemini via n8n)	2 phrases lisibles pour le valideur à partir du JSON des contrôles
3. Enregistrement	`ai_record_review(...)`	verdict `ok` ou `to_check`, tokens, coût en € ; la demande passe en `manager_review`

Chiffrage : chaque appel enregistre `tokens_in`, `tokens_out` et `cost_eur` dans `ai_reviews`. La vue `v_ai_costs_monthly` donne le coût IA réel par mois, ce qui permet de comparer au coût manuel (section 9). Pour une demande de matériel ou de formation, le contrôle indique aussi le niveau requis déduit du montant (manager, RH ou DRH).

Pourquoi cette architecture : un LLM seul peut se tromper sur un solde ou une date ; ici l'erreur est impossible sur les faits, et le LLM ne peut rien casser. Si l'IA est en panne, la demande reste `pending` et le manager peut quand même décider.

8. Supervision, erreurs et migrations

- **Si l'IA ou n8n tombe** : la demande reste `pending`, le manager la voit quand même et peut approuver ou refuser. Aucune demande n'est bloquée par l'automatisation.
- **Erreurs n8n** : les exécutions échouées restent consultables dans n8n ; un workflow d'erreur (Error Trigger + email d'alerte) est la prochaine étape.
- **Traçabilité** : chaque changement de statut est journalé dans `request_history` avec l'auteur, l'heure et la note.
- **Tests** : `supabase/tests/smoke.sql` rejoue tout le parcours (routage, auto-approbation bloquée, solde débité, refus avec motif, visibilités, vues) et se termine par « TOUS LES TESTS PASSENT ».

Migrations (base versionnée dans Git) :

```
supabase login
supabase link --project-ref eljsnrtxyiypqaeirvwd
supabase db push
```

Les 6 fichiers sont horodatés et idempotents ; ajouter une règle = ajouter un fichier, jamais modifier un ancien.

9. Coûts mensuels projetés

Hypothèses (celles de l'énoncé pour le volume et le temps, le reste est à ajuster) : 40 demandes/mois, 10 minutes de traitement manuel par demande, coût horaire chargé de 30 € (hypothèse), environ 600 tokens en entrée et 120 en sortie par contrôle IA.

Poste	Calcul	Coût/mois
Supabase	offre gratuite pour la démo ; offre Pro si mise en production (tarif à vérifier sur supabase.com/pricing)	0 € (démo)
n8n	auto-hébergé (Docker) : 0 € de licence, hébergement en plus en production	0 € (démo)
LLM (Gemini 2.5 Flash)	40 × (600 × 0,30 + 120 × 2,50) / 1 000 000 = environ 0,02 $	< 0,05 €
Total automatisé		**< 1 €** en démo, plus l'abonnement Supabase en production
Traitement manuel	40 × 10 min = 6,7 h × 30 €	**environ 200 €**

Lecture : le LLM coûte quasiment rien à ce volume ; le vrai poste en production est l'hébergement. Même avec un abonnement Supabase Pro, l'économie reste positive dès que le temps de traitement manuel dépasse quelques heures par mois. Le coût réel est mesuré par `v_ai_costs_monthly`.

Source du tarif LLM : Gemini API Pricing (morphllm.com, mis à jour le 25/10/2026) : 0,30 $ / 2,50 $ par million de tokens (entrée / sortie).

10. Plan d'évolution et DORA

Mesure : `v_dora_lead_time` donne le délai moyen, médian et p90 entre création et décision, par type de demande ; `v_stage_durations` montre l'étape la plus lente (goulot).

Indicateur	Avant (manuel, email)	Après (plateforme)
Lead time d'une demande	non mesuré	mesuré en continu (`v_dora_lead_time`)
Goulot d'étranglement	invisible	visible par étape (`v_stage_durations`)
Taux d'erreur de saisie (dates, solde, doublon)	dépend du relecteur	contrôlé automatiquement avant le manager
Traçabilité	emails dispersés	`request_history`

Les valeurs « avant » chiffrées sont à relever par l'équipe : aucune mesure réelle n'existe sur données fictives.

Bascule vers du code sur mesure (par ordre de criticité) :

1. Déjà fait : règles critiques (routage, solde, droits) en SQL testé, pas en low-code.
2. Orchestration IA : passer de n8n à une Edge Function Supabase si le volume ou la latence l'exigent.
3. Notifications : garder n8n tant qu'on ajoute des canaux (Slack, Teams) ; sinon Edge Function.
4. Front : déjà du code (React).

11. Limites et bonus chiffrés

Limites connues : tunnel cloudflared temporaire (URL à changer après chaque redémarrage) ; emails en bac à sable Mailtrap ; workflow d'erreur n8n à configurer ; flux IA n8n à re-tester de bout en bout avec Gemini (le nœud IA a bouclé lors d'un premier essai) ; données fictives.

Bonus proposés (coûts de dev en jours à estimer par l'équipe) :

Bonus	Intérêt utilisateur	Coût de dev	Coût d'exploitation	Critère de décision
Notification Slack/Teams	réponse plus rapide du manager	faible (un nœud n8n)	quasi nul	faire si les managers n'ouvrent pas leurs emails
Export CSV mensuel pour la paie	gain de temps RH	faible	nul	faire si la paie est saisie à la main
Purge RGPD planifiée	conformité	très faible (`purge_old_requests`)	nul	obligatoire en production
Relance automatique des demandes en attente	réduit le lead time	faible	quasi nul	faire si `v_stage_durations` montre un goulot manager